

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Curso de Machine Learning

Profesor Alan Badillo Salas

Práctica 101

Introducción al Machine Learning y las herramientas de trabajo

Semana 1. Enero 29, 2026.

1. Introducción

En esta práctica aprenderemos a elaborar un Proyecto de Machine Learning con Github y Python. Este proyecto servirá de base para las siguientes prácticas y te permitirá entender el proceso en el que se administran proyectos profesionales de Machine Learning.

La práctica consiste en crear un proyecto sencillo que sirva de portafolio para el dominio básico de *Github*, los *Ambientes virtuales en Python* y la visualización de datos con *Pandas* y *Seaborn*.

1.1. Objetivo general

Al finalizar la práctica, el alumno será capaz de crear un proyecto de portafolio en *Github*, que demuestre sus habilidades en *Python*, *Ambientes virtuales*, *Pandas* y *Seaborn*.

1.2. Objetivos particulares

Al finalizar el curso, el alumno será capaz de:

- Comprender que es *Github* y cómo publicar un proyecto
- Registrar cambios en los archivos del proyecto de *Github*
- Publicar los cambios del proyecto de *Github*
- Ver el estado de los cambios del proyecto de *Github*
- Sincronizar los cambios del proyecto de *Github*
- Crear una libreta de *Python* en *Visual Code* y *Jupyter*
- Obtener el conjunto de datos del *titanic.csv*
- Cargar el conjunto de datos del *titanic.csv* como un *DataFrame* de *Pandas*
- Inspeccionar los registros del conjunto de datos del *titanic*
- Construir una variable informativa y una variable de respuesta con los datos del *titanic*
- Visualizar el comportamiento de la variable de respuesta respecto a la variable informativa
- Analizar el patrón de comportamiento entre ambas variables
- Proponer un modelo visual de la mejor curva que se aproxime a los datos
- Concluir los resultados y experiencia generada

1.3. Entregables

La práctica tendrá dos entregables, el primer entregable es un documento PDF creado en cualquier editor de texto que soporte incrustación de imágenes como Word, Excel, Power Point, Canvas o Latex y que permita la exportación o impresión al formato PDF. También se aceptarán trabajos hechos a mano siempre y cuando sean legibles y estén escaneados en formato PDF no mayor a 20mb. A este primer entregable le llamaremos *Documento de la Práctica*

El segundo entregable será el enlace al repositorio de *Github* que deberá estar en su cuenta personal y deberá ser público y accesible por cualquier persona. Este enlace deberá estar contenido en la portada de la práctica del entregable PDF (*Documento de la Práctica*). A este segundo entregable le llamaremos *Repositorio de la práctica*.

Las especificaciones del *Documento de la práctica* son:

- **Portada** - Debe contener el nombre del Instituto Politécnico Nacional, el nombre del Centro de Investigación en Computación (CIC) y el Departamento de diplomados y extensión profesional, así como el logo del IPN a la izquierda y del CIC a la derecha en las esquinas superiores. También deberá decir Curso de Machine Learning y abajo el nombre del profesor, Maestro Alan Badillo Salas y el nombre del alumno (tu nombre). Después pon el nombre del práctica (por ejemplo, Práctica 101 - Introducción al Machine Learning y las herramientas de trabajo), y seguido el enlace al repositorio (por ejemplo, <https://github.com/nemo/practica101-ml-cic-ipn>). Hasta abajo deberá contener la fecha de la práctica o la fecha en que se realizó en el formato: Endidad del instituto. Mes día, año (por ejemplo, Ciudad de México, Febrero 1, 2026).
- **Introducción** - Explica en un párrafo de qué trata la práctica, por ejemplo, en esta práctica se creó un repositorio de *Github* con una libre de *Jupyter Python* que contiene la visualización de datos entre dos variables extraídas del conjunto de datos del *titanic*...
- **Desarrollo** - En el desarrollo deberás explicar la secuencia lógica de la práctica, por ejemplo: Primero creamos un repositorio en github, luego lo clonamos en el escritorio, después lo abrimos con visual code...
- **Resultados** - En esta sección pega las capturas de pantalla que demuestren que creaste el repositorio y su contenido está disponible en *Github*, la captura de pantalla donde se ven los datos del *titanic* en el *DataFrame* de *Pandas*, captura de pantalla donde se muestra la gráfica entre la variable informativa y la variable de respuesta y finalmente el dibujo de Paint o hecha a mano donde ajustas la mejor curva a los datos.
- **Conclusiones** - En las conclusiones deberás resumir que se hizo en general, para qué sirve y qué aplicaciones actuales y futuras tiene (por ejemplo, ¿Se puede aplicar a tu trabajo actual? ¿Qué tipo de proyecto será interesante hacer ahora?).

1.4. Evaluación

La práctica tiene un valor de 100 puntos, de los cuales 50 son suficientes para marcarla como entregada, si tiene menos de 50 puntos no será aceptada y tendrá que ser corregida.

- **20 puntos** - La portada tiene todos los elementos
- **10 puntos** - El enlace del repositorio está en la portada y dirige al repositorio de *Github*
- **10 puntos** - El repositorio de *Github* contine un archivo *README.md* con los datos del instituto, el profesor, el alumno, la introducción y resultados
- **5 puntos** - El repositorio de *Github* contine la libreta de python y el PDF de la práctica
- **5 puntos** - La práctica contine una Introducción razonable
- **10 puntos** - La práctica contine la secuencia lógica de pasos completa y es razonable
- **20 puntos** - La práctica contine los resultados solicitados
- **10 puntos** - La práctica contine conclusiones razonables
- **10 puntos** - La práctica fue entregada antes de la práctica siguiente

Si por razones de tiempo no quieren ajustar el archivo *README.md* del repositorio consideren que alcanzarán 90 puntos máximo, suficientes para marcar la práctica como entregada. La calificación final del alumno se basará en la cantidad de prácticas entregadas, no en su puntaje acumulado. Por lo que, para los que tienen una alta carga de trabajo semanal, pueden optar por cubrir los 50 puntos requeridos, pero la experiencia de completar las prácticas les dará habilidades profesionales que los diferenciarán al encontrar trabajos y mostrar sus portafolios. No será lo mismo poseer el conocimiento (50 puntos) que demostrar su dominio (alcanzar más de 90 puntos).

2. Teoría

2.1. Introducción a Github

Git es un sistema de repositorios de códigos que permite almacenar los cambios ocurridos en el código fuente y archivos de un proyecto. Fue diseñado para permitir que múltiples usuarios puedan colaborar puntualmente en un proyecto de tal manera que se logren resolver las diferencias entre los cambios que hace un colaborador y otro. Este sistema es muy utilizado para publicar y mantener el código en la nube mediante plataformas como *Github*.

Github es actualmente el más utilizado para compartir y publicar proyectos científicos y de software a modo de repositorios públicos y privados que permiten destacar al usuario en los proyectos que ha realizado, por ejemplo, muchos Analistas de datos y Científicos de datos utilizan *Github* como portafolio profesional para demostrar los proyectos en los que ha participado.

La estructura general de un proyecto de Github es:

- El archivo principal de información llamado *README.md*
- Las carpetas y códigos que se deseen compartir
- El archivo que define como ignorar otros archivos llamado *.gitignore*
- Las carpetas con recursos estáticos como imágenes de logos y capturas de pantalla

En el archivo *README.md* usaremos texto con formato *Markdown* para explicar el proyecto, podemos hacer referencia a los logos de la empresa o institución que respalda nuestra investigación, el nombre de los investigadores y sus contactos, la introducción al proyecto y los resultados que generalmente son capturas de pantalla.

Clonar un repositorio en el Escritorio

```
Desktop$ git clone https://github.com/<usuario>/<repo>
```

Sincronizar los cambios del repositorio

```
repo$ git pull
```

Agregar todos los archivos nuevos

```
repo$ git add .
```

Confirmar los cambios

```
repo$ git commit -m "..."
```

Subir los cambios al repositorio

```
repo$ git push
```

2.2. Introducción a los entornos virtuales

Al instalar *Python* las librerías científicas y útiles quedan en la computadora local de tal manera que podemos trabajar comodamente con ellas en cualquier proyecto, configuradas permanentemente. Sin embargo, cuando trabajamos en Proyectos de *Machine Learning*, generalmente publicamos nuestros proyectos ya sean académicos o empresariales, olvidando las versiones de los paquetes que instalamos y que funcionaban adecuadamente en nuestro proyecto. Esto hace que los proyectos queden inservibles al paso de los años y no podemos hacer una demostración de su funcionamiento. El caso más peligroso es cuando intentamos vender un proyecto que trabajamos hace un par de años y el cliente requiere que adaptemos los mismos resultados a nueva información. Aquí es cuando las versiones ya no son compatibles y no logramos hacerlo funcionar.

Es por esto, que cada proyecto de *Machine Learning* debería funcionar con su propio ambiente virtual, y las librerías que funcionen ahora sean suficientes para publicar el proyecto y continuarlo en el futuro. Una contra es que cada proyecto ocupará un par de gigabytes de almacenamiento, pero la gran ventaja es que podremos publicar y almacenar el proyecto en *Github* y luego reconstruirlo en cualquier momento. Es decir, el ambiente virtual puede ser borrado fácilmente para no ocupar almacenamiento cuando se termine el proyecto y reconstruido cuando sea necesario retormarlo.

Crear un ambiente virtual

```
p101$ python -m venv <ml-env>
```

Entrar al ambiente virtual

```
# WINDOWS

p101$ <ml-env>\Scripts\activate

# UNIX/LINUX

p101$ <ml-env>/bin/activate
```

Instalar paquetes en el ambiente virtual

```
(ml-env)$ pip install <paquete>
```

Congelar los paquetes instalados del ambiente virtual

```
(ml-env)$ pip freeze > requirements.txt
```

Reinstalar los paquetes congelados en el ambiente virtual

```
(ml-env)$ pip install -r requirements.txt
```

Salir del ambiente virtual

```
(ml-env)$ deactivate
```

2.3. Introducción a Pandas y Seaborn

En *Python* existen muchas librerías para el manejo de datos y la visualización de datos, dentro de las cuales las más populares son *Pandas* para cargar a la memoria conjuntos de datos en forma de tabla y *Seaborn* para generar gráficas sencillas y muy estilizadas útiles para reportes científicos y análisis de datos.

En *Pandas* una tabla de datos es conocida como un *DataFrame*, el cual es un objeto que se puede definir manualmente desde listas y diccionarios de *Python* o cargarse desde archivos de texto separados por coma (*CSV*) o *Excel*, aunque también soporta *SQL*, *JSON*, *HTML* y otros formatos.

Un *DataFrame* es un objeto complejo que puede recuperar la información de las filas como *Series* de datos y también generar reportes de datos concentrados y visualizaciones directas como histogramas, barras y líneas.

Por otro lado, si pensamos las columnas de la tabla de datos como *Series* o ejes independientes, podemos almacenarlos en variables para usarlos en gráficas más avanzadas mediante la librería de *Seaborn*.

Algunas manipulaciones útiles de ambas librerías son:

Importar la librería de Pandas y Seaborn

```
import pandas
import seaborn
```

Cargar un CSV con Pandas

```
titanic = pandas.read_csv(url)
```

- La *url* puede ser la ruta local o web.

Mostrar la información del DataFrame

```
titanic.info()
```

- Describe las columnas, número de registros, nombre y tipo de datos en cada columna, número de valores faltantes y memoria utilizada.

Mostrar las estadísticas básicas del DataFrame

```
titanic.describe()
```

- Describe las columnas numéricas estadísticamente: total, media, mínimo, máximo, desviación estándar y cuartiles.

Mostrar los primeros registros del DataFrame

```
titanic.head()
```

- Describe las primeras 5 filas o el número deseado si se especifica.

Mostrar los últimos registros del DataFrame

```
titanic.tail()
```

- Describe las últimas 5 filas o el número deseado si se especifica.

Recuperar una columna como una Series de datos

```
x1 = titanic["age"]  
x2 = titanic["sex"]  
y = titanic["survived"]
```

- La columna representa una *Series* de *Pandas*.

Recuperar una columna como una Series de datos lógica

```
x3 = titanic["age"] > 30  
x4 = titanic["sex"] == "female"  
x5 = titanic["sex"] == "male"  
x6 = titanic["pclass"] == 1  
x7 = titanic["pclass"] == 2  
x8 = titanic["pclass"] == 3
```

- Las variables lógicas contienen *True* y *False* en lugar de valores numéricos.

Recuperar una columna como una Series de datos binaria 0 o 1

```
x3 = (titanic["age"] > 30).astype(int)  
x4 = (titanic["sex"] == "female").astype(int)  
x5 = (titanic["sex"] == "male").astype(int)  
x6 = (titanic["pclass"] == 1).astype(int)  
x7 = (titanic["pclass"] == 2).astype(int)  
x8 = (titanic["pclass"] == 3).astype(int)
```

- Las variables lógicas se pueden convertir a 1 si es *True* o 0 si es *False*.

Visualizar una gráfica de puntos entre x_1 y y con Seaborn

```
x1 = titanic["age"]
y = titanic["fare"]

seaborn.scatterplot(x=x1, y=y)
```

- La gráfica de puntos permite ver la correlación entre dos ejes de datos.

Visualizar una gráfica de puntos entre x_1 y y segmentada por g con Seaborn

```
x1 = titanic["age"]
y = titanic["fare"]

g = titanic["sex"]

seaborn.scatterplot(x=x1, y=y, hue=g)
```

- La gráfica de puntos puede ser segmentada por un tercer eje de datos, mostrando diferentes colores para cada grupo marcado por g . Se recomienda que la variable de segmentación sea categórica o discreta para mejor apreciación.

3. Desarrollo

3.1. Práctica 101

Usando la teoría expuesta sigue los pasos para completar la práctica:

1. Crea una cuenta de *Github*
2. Dentro de *Github* crea un nuevo repositorio público llamado *p101* o el nombre adecuado (por ejemplo, *practica101-ml-cic-ipn-feb-2026*). Genera una buena descripción y activa el archivo *README.md*
3. Clona el repositorio en el escritorio mediante la consola (terminal o símbolo del sistema) o usando *Github Desktop* (recomendado)
4. Abre la carpeta del repositorio clonado con *Visual Code*
5. Crea un archivo llamado *.gitignore* y agrega la línea de texto *ml-env/* para que ignore al ambiente virtual y no lo suba al repositorio
6. Edita el archivo *README.md* usando títulos, subtítulos e imágenes locales al repositorio o desde la URL para crear la portada del repositorio (no es la portada del documento de la práctica).
7. Abre la terminal integrada en *Visual Code* con *CTRL+J* en *Windows* o *CMD+J* en *MacOS*
8. Dentro de la terminal crea un entorno virtual llamado *ml-env* o a tu gusto
9. Activa el entorno virtual (ingresa al *prompt* del ambiente virtual)
10. Instala el paquete llamado *pandas* y el paquete llamado *seaborn*

11. Congela los requerimientos de estos paquetes en el archivo *requirements.txt*
12. No salgas del entorno virtual
13. Crea un archivo llamado *test.ipynb* para crear una libreta de *Python* con *Jupyter*
14. Abre el archivo *test.ipynb* y sigue las instrucciones para instalar *Jupyter* y lo que haga falta
15. Cuando esté todo instalado verifica que el Kernel de la libreta de *Python* sea el entorno virtual
16. Escribe en una celda de código $2 + 2$ y ejecuta la celda. Debería mostrar 4 como resultado
17. Crea otra libreta llamada *p101.ipynb* o como te sea mejor el nombre
18. En una celda de código importa las librerías de *pandas* y *seaborn*
19. Descarga el archivo *CSV* con los datos del *Titanic*
20. Pon el archivo *titanic.csv* en la carpeta del repositorio a lado de las libretas
21. En la libreta *p101.ipynb* en una celda de código carga el *DataFrame* como la variable *titanic*
22. Muestra la información de *titanic*
23. Muestra la descripción estadística de *titanic* e interpreta su significado
24. Muestra las primeras 10 filas del *titanic*
25. Muestra las últimas 10 filas del *titanic*
26. Crea la variable x_1 con la columna *age* (edad)
27. Crea la variable y con la columna *fare* (tarifa)
28. Crea la variable g con la columna *sex* (sexo)
29. Grafica los puntos entre x_1 y y usando *Seaborn*
30. Grafica los puntos entre x_1 y y segmentados por g usando *Seaborn*
31. Saca una captura de pantalla y pégala en *Paint* o *Canvas*, o alternativamente, copia la gráfica en una libreta u hoja milimétrica y con un color que resalte ajusta una curva que se apegue lo mejor posible a los datos siendo los suficientemente flexible y rígida a la vez.
32. Guarda la libreta
33. Usa los controles de repositorio de *Visual Code* o *Github Desktop* (o más avanzado desde la terminal), para confirmar los cambios del repositorio y subirlos
34. Verifica que los cambios estén en *Github* (saca captura de pantalla para reportarlo)
35. Genera el *Documento de la Práctica* en *Word* o algún editor de texto utilizando capturas de pantalla para demostrar tus resultados
36. Edita el archivo *README.md* para actualizar los resultados de este proyecto y súbelos a *Github*